
I

INTRODUCTION

It doesn't take a huge amount of knowledge and skill to get a program working. Kids in high school do it all the time. Young men and women in college start billion-dollar businesses based on scrabbling together a few lines of PHP or Ruby. Hoards of junior programmers in cube farms around the world slog through massive requirements documents held in huge issue tracking systems to get their systems to "work" by the sheer brute force of *will*. The code they produce may not be pretty; but it works. It works because getting something to work—once—just isn't that hard.

Getting it right is another matter entirely. Getting software right is *hard*. It takes knowledge and skills that most young programmers haven't yet acquired. It requires thought and insight that most programmers don't take the time to develop. It requires a level of discipline and dedication that most programmers never dreamed they'd need. Mostly, it takes a passion for the craft and the desire to be a professional.

And when you get software right, something magical happens: You don't need hordes of programmers to keep it working. You don't need massive requirements documents and huge issue tracking systems. You don't need global cube farms and 24/7 programming.

When software is done right, it requires a fraction of the human resources to create and maintain. Changes are simple and rapid. Defects are few and far between. Effort is minimized, and functionality and flexibility are maximized.

Yes, this vision sounds a bit utopian. But I've been there; I've seen it happen. I've worked in projects where the design and architecture of the system made it easy to write and easy to maintain. I've experienced projects that required a fraction of the

anticipated human resources. I've worked on systems that had extremely low defect rates. I've seen the extraordinary effect that good software architecture can have on a system, a project, and a team. I've been to the promised land.

But don't take my word for it. Look at your own experience. Have you experienced the opposite? Have you worked on systems that are so interconnected and intricately coupled that every change, regardless of how trivial, takes weeks and involves huge risks? Have you experienced the impedance of bad code and rotten design? Has the design of the systems you've worked on had a huge negative effect on the morale of the team, the trust of the customers, and the patience of the managers? Have you seen teams, departments, and even companies that have been brought down by the rotten structure of their software? Have you been to programming hell?

I have—and to some extent, most of the rest of us have, too. It is far more common to fight your way through terrible software designs than it is to enjoy the pleasure of working with a good one.